WORKSHOP BUILD GUIDE

Build Your AI Project Manager

Take-home reference: the concepts, every code step, and how to run it — CrewAI + local Ollama.

This is your companion for the hands-on session and your reference afterward. The runnable files (pm_crew.py, tool_demo.py, virtual_office.py) are in the code bundle, and every build stage has a working file in checkpoints/ if you fall behind; this guide explains what each step does and why.

The mental model

- **Agent —** an LLM plus tools, memory, and a loop. Strip the hype and that's all it is.
- **ReAct loop —** each turn the agent Thinks, takes an Action (calls a tool), Observes the result, and repeats until the goal is met. CrewAI runs this loop for you.
- **Crew —** one agent for a focused job; a crew of specialists for complex work. Sequential = a pipeline; a Lead can also delegate to specialists.
- **Local stack —** CrewAI orchestrates, talks to Ollama directly, Ollama runs open weights (Llama 3.1 / Llama 3.2) on your machine. No API keys, no data leaving the laptop.

Setup

Install a local model once, create a clean Python environment, and add the framework.

```
# 1. install Ollama (ollama.com), then pull open models
ollama pull llama3.1          # ~4.7 GB, runs fully local
ollama pull llama3.2:3b       # ~2 GB, for weaker laptops

# 2. a clean Python 3.10-3.13 environment + the framework
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# 3. sanity check: is everything ready?
python check_setup.py
```

Build the PM assistant, step by step

1. Your first agent

An Agent, a Task, and a Crew to run it — pointed at the local Ollama model.

```
from crewai import Agent, Task, Crew, Process, LLM

# point CrewAI at the LOCAL model served by Ollama
llm = LLM(model="ollama/llama3.1",
          base_url="http://localhost:11434")

planner = Agent(
    role="Project Planner",
```

```
    goal="Turn a project brief into a clear task list",
    backstory="A sharp PM who makes vague briefs actionable.",
    llm=llm, verbose=True,
)

task = Task(
    description="Break down this brief into tasks:\n{brief}",
    expected_output="A numbered task list with an owner each",
    agent=planner,
)

crew = Crew(agents=[planner], tasks=[task], process=Process.sequential)
print(crew.kickoff(inputs={"brief": open("brief.md").read()}))
```

Why: One agent + one task + a crew is a working assistant in about 20 lines.

2. Give it a tool

Tools are plain Python functions. If the model decides it needs one, CrewAI calls it and feeds the result back.

```
from crewai.tools import tool
import json

@tool("save_tasks", max_usage_count=1)
def save_tasks(tasks_json: str) -> str:
    "Save a JSON list of task titles to tasks.json."
    data = json.loads(tasks_json)
    with open("tasks.json", "w") as f:
        json.dump(data, f, indent=2)
    return f"saved {len(data)} tasks"

# the agent may now decide to call this tool
planner.tools = [save_tasks]
```

Why: The model decides whether to call a tool, so it may skip it. Run tool_demo.py a few times to see this. Expose only the tools an agent truly needs (least privilege).

3. Typed output, not prose

Make the planner return a validated Pydantic object so the next step can trust the data.

```
from pydantic import BaseModel
from typing import List

class TaskItem(BaseModel):
    title: str
    owner: str
    estimate_days: int

class Plan(BaseModel):
    tasks: List[TaskItem]

plan_task = Task(
    description="Break the brief into tasks:\n{brief}",
    expected_output="A Plan object",
    agent=planner,
    output_pydantic=Plan,     # validated, typed output
)
```

Why: Typed output is what makes agents composable — no regex-parsing of prose.

4. Add two specialists

A risk analyst and a status writer, each with a single, narrow responsibility.

```
risk = Agent(
    role="Risk Analyst",
    goal="Flag the top risks, with one mitigation each",
    backstory="A cautious PM who has watched projects slip.",
    llm=llm)

writer = Agent(
    role="Status Writer",
    goal="Draft a short status update for stakeholders",
    backstory="Writes clear, calm updates and never invents facts.",
    llm=llm)

risk_task = Task(description="Review the plan. List the top risks.",
                 expected_output="A risk register", agent=risk,
                 context=[plan_task])

status_task = Task(description="Write a kickoff update. Work has NOT started: claim no
progress.",
                   expected_output="At most 5 lines", agent=writer,
                   context=[plan_task, risk_task])
```

Why: Pass context between tasks with context=[...] so each agent sees the previous output.

5. Run the PM crew

Wire the three agents into one sequential crew and run it on a real brief.

```
from pathlib import Path

crew = Crew(
    agents=[planner, risk, writer],
    tasks=[plan_task, risk_task, status_task],
    process=Process.sequential,     # planner -> risk -> writer
    verbose=True,
)

result = crew.kickoff(inputs={"brief": open("brief.md").read()})
print(result)      # the final task's output: the status update

# save shared state from Python, so it can't depend on the model's choice
plan = result.tasks_output[0].pydantic
Path("project_state.json").write_text(plan.model_dump_json(indent=2))
```

Why: That's your AI project manager — in about 40 lines. print(result) shows only the final task's output; result.tasks_output holds every task's.

Intermezzo: the virtual office (NexaPaw)

Switch from a fixed pipeline to a named Lead that returns a typed work plan, which Python dispatches to a specialist team — the pattern behind Felix's NexaPaw office, where every Claude Code role is an agent (a CEO and CTO above project leads above specialist engineers), visualized live in a 3D HQ. You define the staff; the Lead decides who does what, and when.

```
def staff(role, goal, backstory):
    return Agent(role=role, goal=goal, backstory=backstory, llm=llm, verbose=True)

lead = staff("Project Lead", "Assign work to the right specialist", "Runs a NexaPaw project
team.")
```

```
team = {
    "CV Engineer": staff("CV Engineer", "Build and evaluate the model", "A pragmatic CV
engineer."),
    "Backend Engineer": staff("Backend Engineer", "Build the APIs and data pipeline", "A
careful backend engineer."),
    "Frontend Engineer": staff("Frontend Engineer", "Build the user interface", "A tidy
frontend engineer."),
    "QA Engineer": staff("QA Engineer", "Test the work against the spec", "A sharp, fair
tester."),
}

class Assignment(BaseModel):     # a role must name a real staff member
    role: Literal["CV Engineer", "Backend Engineer", "Frontend Engineer", "QA Engineer"]
    work: str

class WorkPlan(BaseModel):
    assignments: List[Assignment]

# 1. the Lead decides who does what
planning = Task(description=f"Assign this goal to your team ({', '.join(team)}): {brief}",
                expected_output="A WorkPlan with at most 3 assignments", agent=lead,
                output_pydantic=WorkPlan)
plan = Crew(agents=[lead], tasks=[planning]).kickoff().pydantic

# 2. Python dispatches the Lead's decisions to the specialists
tasks = [Task(description=f"{a.work}. Use at most 4 bullet points.", agent=team[a.role],
              expected_output="At most 4 bullet points") for a in plan.assignments]
result = Crew(agents=list(team.values()), tasks=tasks).kickoff()
print(result)
```

HONEST TRADE-OFF

A virtual office is powerful and harder to control. In testing on Llama 3.1 and 3.2, Process.hierarchical never reached a specialist: it exited cleanly with delegation written as text, so this build uses typed delegation. Always check who actually ran. More agents = more model calls = more time and nondeterminism. Reach for it only when a single agent plateaus.

Make it real

Validate outputs, cap iterations, and keep consequential actions behind a person.

```
# guardrail: Python validates the output; failures go back to the agent
def short_update(output: TaskOutput) -> tuple[bool, Any]:
    if len(output.raw.split()) > 120:
        return (False, "Too long. Keep it under 120 words.")
    return (True, output.raw)

status_task = Task(description="...", expected_output="...", agent=writer,
                   guardrail=short_update, guardrail_max_retries=2)

# gate risky actions behind a human — draft, never send
@tool("send_email")
def send_email(to: str, body: str) -> str:
    "Save a draft for human approval; does NOT send."
    Path("outbox.md").write_text(body)
    return "DRAFT saved to outbox.md — needs approval"

# stop a stuck agent from looping forever
planner = Agent(role="Project Planner", goal="...", backstory="...",
                llm=llm, max_iter=8, max_retry_limit=2)
```

- Turn on verbose / logging to see each thought, tool call, and result.
- Trace tokens and latency — local models are free but slower; cache repeated calls.
- Evaluate on a fixed set of briefs with known-good outputs before you trust it.
- Local-first keeps customer data off third-party servers (helps with PDP Law).
- Watch for prompt injection when agents read external text, email, or web pages.

SIMPLE RULE

An agent should propose; a person should dispose — for anything that matters (send, pay, delete, or decisions about people).

Run the finished code

- ollama pull llama3.1 (once)
- pip install -r requirements.txt
- python pm_crew.py --brief brief.md — plans tasks, flags risks, drafts a status update
- python tool_demo.py — one agent choosing whether to call a tool
- python virtual_office.py — the Lead plans, then the specialists run. Weak laptop? WORKSHOP_MODEL=ollama/llama3.2:3b python virtual_office.py

---

Prepared by Felix Yustian Setiono for the MurzEdu Program (ecommurz). Code is a starting point — adapt it to your stack.